

MODEL CONTEXT PROTOCOL · DEEP DIVE

Module 1

MCP Fundamentals & Architecture

Why MCP exists · participants · two-layer architecture · primitives · the lifecycle on the wire · versioning

1 HOUR

THEORY + LAB

INSTRUCTOR-LED

What This Module Covers

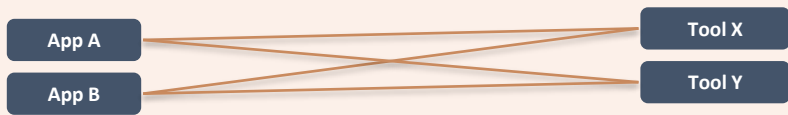
- 1 Why MCP Exists** — The N×M integration problem & “USB-C for AI”
- 2 Participants** — Host, Client, Server — local vs remote
- 3 Two-Layer Architecture** — Data layer (JSON-RPC 2.0) + Transport layer
- 4 Primitives** — Tools / Resources / Prompts · Sampling / Elicitation / Logging
- 5 Lifecycle on the Wire** — initialize → discover → execute → notify
- 6 Versioning & Negotiation** — Date-based revisions; behaviour on mismatch
- 7 Hands-On Lab** — Observe the protocol live with the MCP Inspector

Why MCP Exists — The N×M Integration Problem

MCP is an **open standard for connecting AI applications to external systems** — data sources, tools, and workflows — so models can access information and take actions.

- ▶ **The problem:** Every AI app needed a custom, one-off connector per external system.
 - N apps × M systems → **N×M** bespoke integrations to build, test, secure, version & maintain.
 - Add one tool → re-integrate into every app; add one app → re-integrate every tool.
- ▶ **MCP collapses this to N + M:** each app implements the protocol once (host/client), each system once (server).
 - Any compliant client can talk to any compliant server — build once, integrate everywhere.

BEFORE — N×M tangle



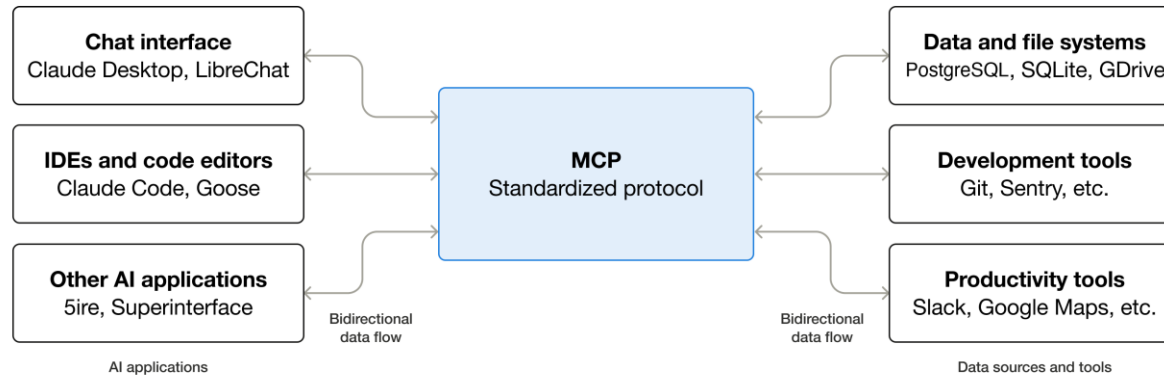
AFTER — N+M via MCP



The Mental Model: “USB-C for AI”

Just as USB-C provides one standardized way to connect devices, MCP provides one standardized way to connect AI applications to external systems.

The canonical “AI app ↔ MCP ↔ external systems” concept diagram:



What MCP Enables & Why It Matters

What it enables

- ▶ Agents that reach your Calendar & Notion as a personalized assistant
- ▶ Claude Code generating a web app from a Figma design
- ▶ Enterprise chatbots querying many databases via chat
- ▶ AI creating 3D designs in Blender, then printing them

Who benefits

Audience	Benefit
Developers	Less time & complexity building AI integrations
AI apps / agents	Instant ecosystem of data, tools & apps
End-users	More capable AI that acts on their behalf

Scope note: MCP focuses solely on the protocol for context exchange. It does not dictate how apps use LLMs or manage the provided context.

Participants: Host, Client, Server

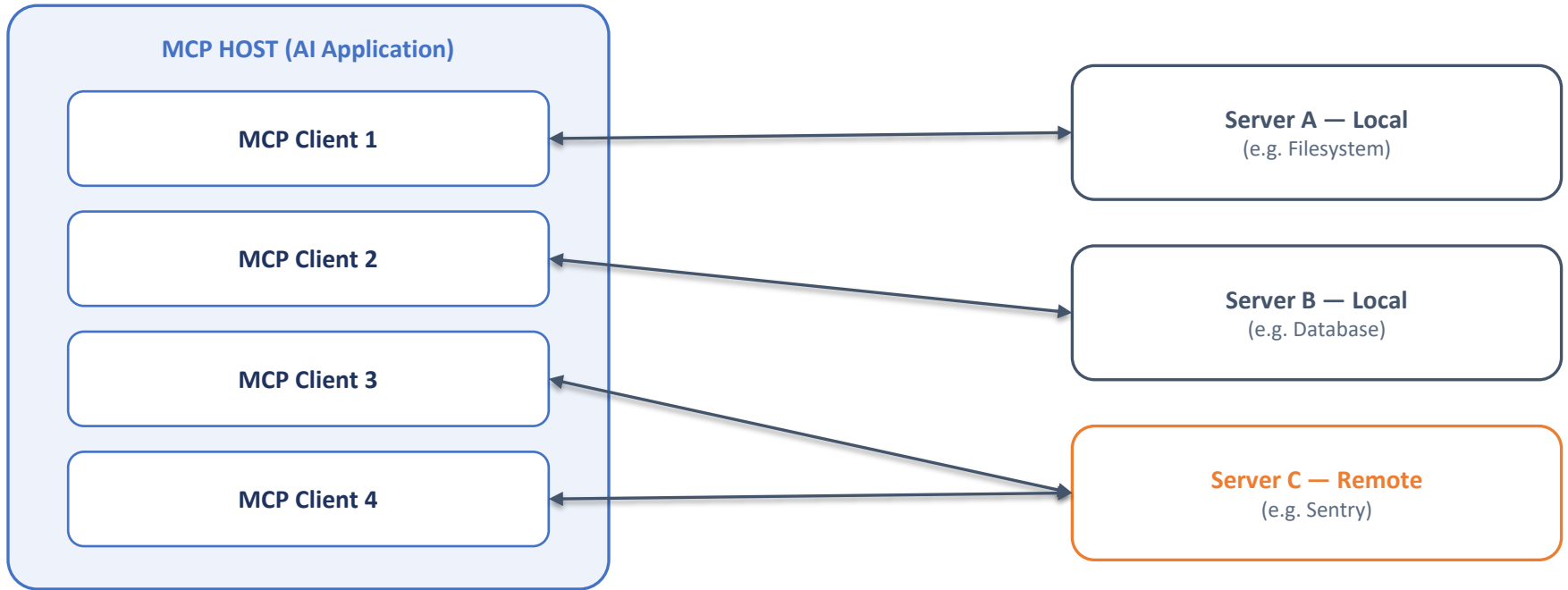
Participant	Definition
MCP Host	The AI application that coordinates & manages one or many MCP clients
MCP Client	Maintains a connection to one server and obtains context for the host
MCP Server	A program that provides context (tools, resources, prompts) to clients

One host → many clients → each client bound 1:1 to one server.

Worked example (from the spec):

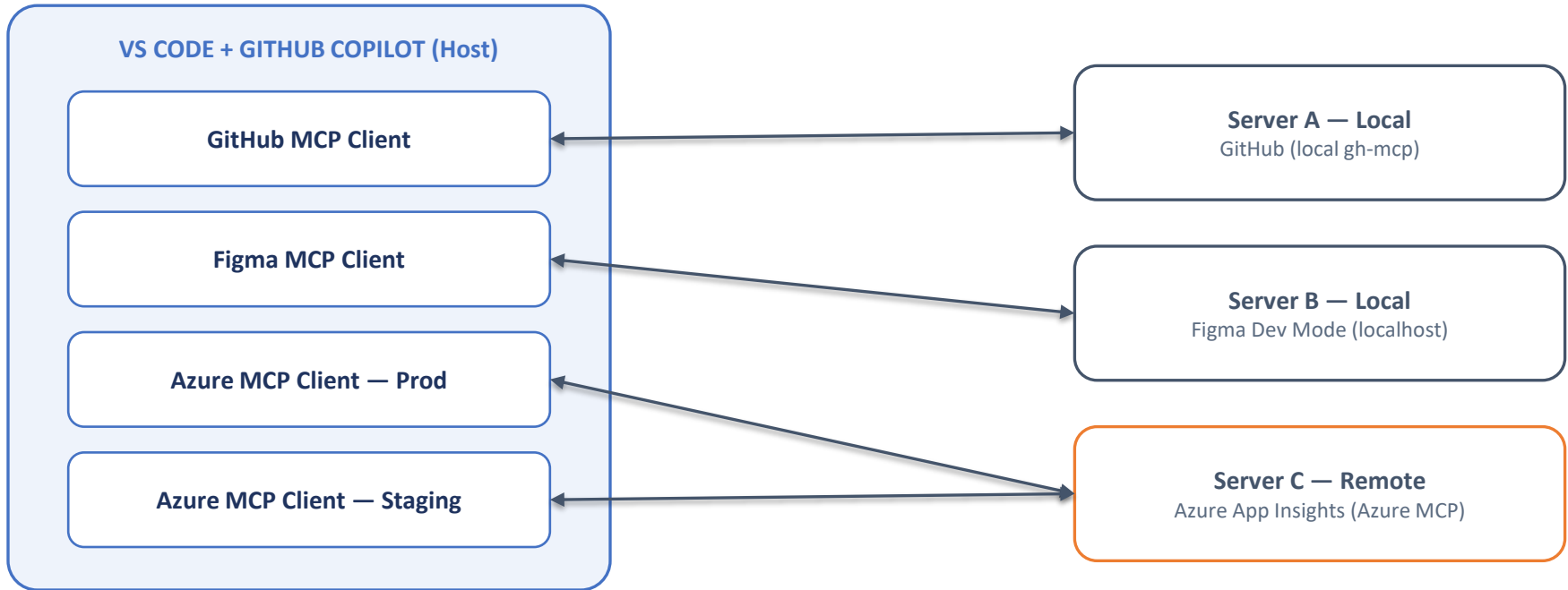
VS Code is the host. Connecting to the Sentry server instantiates one client; later connecting to the local filesystem server instantiates an additional client — each maintaining its own 1:1 connection.

Host ↔ Clients ↔ Servers



Clients 3 and 4 each hold their own dedicated connection to the same remote Server C — the hallmark of a multi-client remote server.

VS Code ↔ Copilot Clients ↔ Dev Tool Servers



Two Azure MCP clients (prod & staging) each hold their own dedicated connection to the same remote Azure MCP Server — querying Application Insights telemetry across environments from VS Code.

Local vs Remote Servers

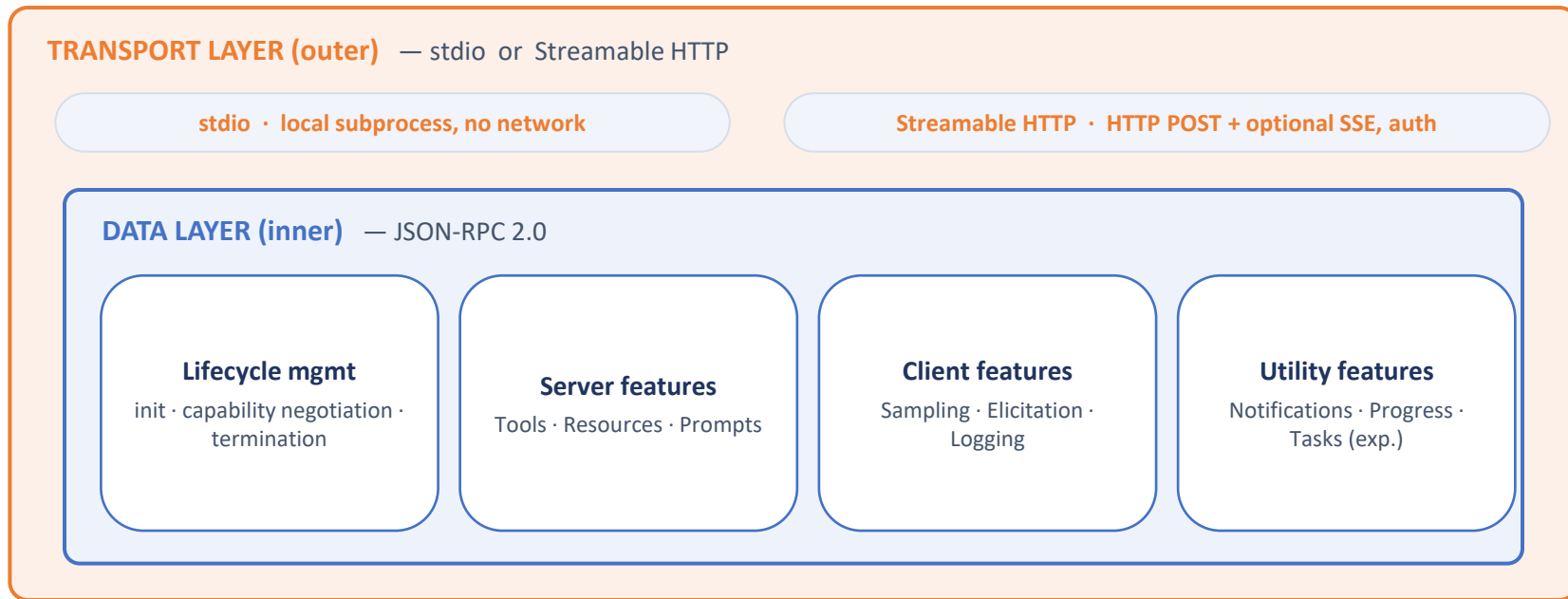
LOCAL SERVER

- ▶ Runs on the **same machine**, launched by the host
- ▶ Communicates over the **STDIO transport**
- ▶ Launched as subprocesses directly by the host runtime.
- ▶ Typically serves a single MCP client
- ▶ **Example:** Claude Desktop launching the filesystem server

REMOTE SERVER

- ▶ Runs on a **remote platform**
- ▶ Communicates over **Streamable HTTP**
- ▶ Typically serves many MCP clients
- ▶ **Example:** the Sentry MCP server on the Sentry platform

The Two-Layer Architecture



The same JSON-RPC 2.0 messages (inner box) flow unchanged regardless of which transport wraps them.

Data Layer & Transport Layer

Data layer — the inner layer (JSON-RPC 2.0)

- ▶ **Lifecycle management** — initialization, capability negotiation, termination
- ▶ **Server features** — tools (actions), resources (context), prompts (templates)
- ▶ **Client features** — sampling, elicitation, logging
- ▶ **Utility features** — notifications & progress; MCP is a stateful protocol

Transport layer — the outer layer

- ▶ **Stdio transport** — std in/out between local processes; optimal performance, no network
- ▶ **Streamable HTTP** — HTTP POST (+ optional SSE); supports bearer tokens, API keys; MCP recommends OAuth
- Abstracts communication away from the protocol — same message format on every transport (see Module 8)

The Two Transports, Side by Side

Same JSON-RPC messages, very different operational properties — pick the transport by where the server has to live.

Property	stdio	Streamable HTTP
Who starts the server	The client launches it as a subprocess	Runs independently — you start it
Channel	stdin / stdout, newline-delimited JSON	HTTP POST + optional GET/SSE, one endpoint
Lifetime	Dies with the client	Long-lived; serves many clients
Concurrency	One client, one process	Many clients, one process
Auth	Inherits the user's machine identity	Needs OAuth / tokens (Module 10)
Secrets	Env vars in the host config	Server-side env or key vault
Scaling	Not applicable	Load balancer, multiple replicas
Debugging	stderr + Inspector	HTTP logs, curl, Inspector
Best for	Local dev tools, filesystem, git, local DBs	Shared / team servers, SaaS, anything remote

Rule of thumb: local tool on the user's machine → stdio. Shared, remote or multi-user → Streamable HTTP. The data layer is identical either way.

Two Things Worth Saying Out Loud

Two points students consistently get wrong about transports.

1 · stdio is the default, not the fallback

The specification says clients **SHOULD support stdio whenever possible**. Treat it as the first choice for anything local — filesystem, git, local databases, dev tooling — and reach for Streamable HTTP only when the server genuinely has to live somewhere else.

2 · HTTP+SSE is legacy — Streamable HTTP replaced it

The old two-endpoint HTTP+SSE transport (protocol revision 2024-11-05) is **deprecated**, replaced by Streamable HTTP on a single endpoint. You will still meet **“type”: “sse”** in older blog posts and host configs — recognise it as legacy when it shows up on the host-config slides in Modules 2 and 3.

Primitives — Exposed by Servers

Primitives define what clients and servers offer each other — the most important concept in MCP.

Primitive	What it is	Example	Key methods
Tools	Executable functions the AI can invoke	API calls, DB queries	tools/list · tools/call
Resources	Data sources that provide context	File contents, records	resources/list · resources/read
Prompts	Reusable interaction templates	System prompts, few-shot	prompts/list · prompts/get

Discovery is live: clients call the */list methods to discover primitives, so listings can be **dynamic**. A DB server might expose query tools, a schema resource, and a few-shot prompt. Server primitives are built hands-on in Module 5.

Primitives — Exposed by Clients

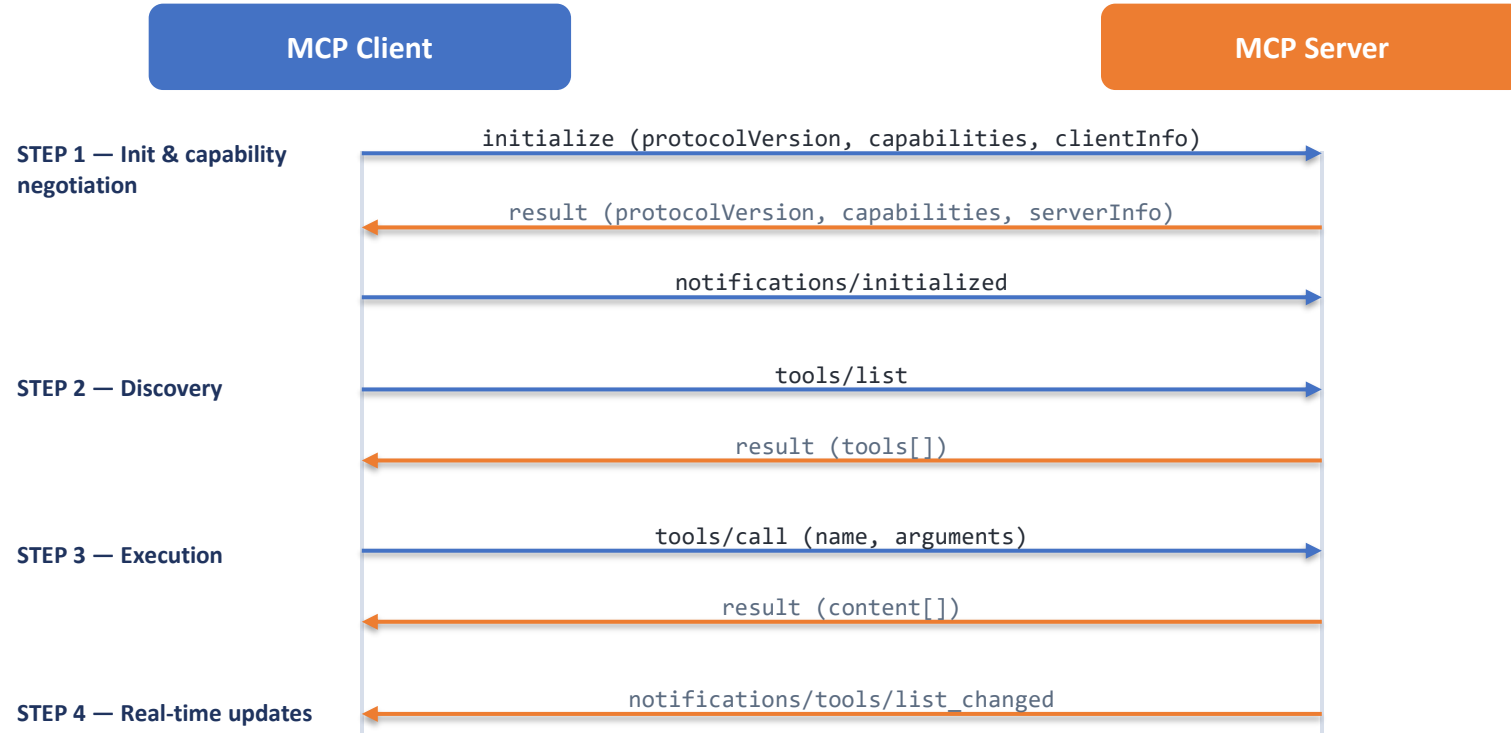
Servers build richer interactions using primitives the client exposes:

Primitive	What it is	Method
Sampling	Server requests an LLM completion from the client's app	sampling/createMessage
Elicitation	Server requests more input from the user mid-interaction	elicitation/create
Logging	Server sends log messages to the client	log notifications

Cross-cutting utility — Tasks (Experimental): durable execution wrappers for deferred result retrieval & status tracking (expensive computations, batch jobs, multi-step workflows).

Client primitives are explored hands-on in Module 9.

The Lifecycle on the Wire



Step 1 — Initialization & Capability Negotiation

// Initialize Request

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "elicitation": {}
    },
    "clientInfo": {
      "name": "example-client",
      "version": "1.0.0"
    }
  }
}
```

// Initialize Response

```
{
  "jsonrpc": "2.0", "id": 1,
  "result": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "tools": {
        "listChanged": true
      },
      "resources": {}
    },
    "serverInfo": {
      "name": "example-server",
      "version": "1.0.0"
    }
  }
}
```

- ▶ **Version negotiation** — both sides must agree on one compatible protocolVersion
- ▶ **Capability discovery** — each party declares what it supports (e.g. tools.listChanged)
- ▶ **Identity exchange** — clientInfo / serverInfo for debugging; then notifications/initialized

Step 2 — Tool Discovery

```
// Tools List Request
```

```
{  
  "jsonrpc": "2.0",  
  "id": 2,  
  "method": "tools/list"  
}
```

- ▶ **name** — unique id in the server's namespace
- ▶ **title** — human-readable display name
- ▶ **description** — what it does / when to use it
- ▶ **inputSchema** — JSON Schema of expected parameters

```
// Tools List Response
```

```
{ "jsonrpc": "2.0", "id": 2,  
  "result": { "tools": [  
    { "name": "calculator_arithmetic",  
      "title": "Calculator",  
      "description": "Perform math ...",  
      "inputSchema": {  
        "type": "object",  
        "properties": {  
          "expression": { "type": "string" } },  
        "required": ["expression"] } },  
    { "name": "weather_current", ... }  
  ] }  
}
```

Step 3 — Tool Execution

// Tool Call Request

```
{
  "jsonrpc": "2.0",  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "weather_current",
    "arguments": {
      "location": "San Francisco",
      "units": "imperial"
    }
  }
}
```

// Tool Call Response

```
{
  "jsonrpc": "2.0",  "id": 3,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Current weather in
                San Francisco: 68°F,
                partly cloudy ..." }
    ]
  }
}
```

The client sends **tools/call** with a tool name + arguments; the server returns a structured **content[]** array. You'll watch this exact round-trip in the lab.

Step 4 — Real-Time Updates (Notifications)

```
// Server → Client notification

{
  "jsonrpc": "2.0",
  "method":
    "notifications/tools/list_changed"
}
```

On receiving it, the client typically re-fetches the list:

```
// Client re-requests the updated list

{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "tools/list"
}
```

- ▶ **No response required** — a notification has **no id field**
- ▶ **Capability-based** — only sent by servers that declared `listChanged: true`
- ▶ **Event-driven** — the server decides when, based on internal state

This keeps the host's view of available capabilities in sync with a server whose tools change at runtime.

Versioning & Negotiation

MCP uses string version ids in the format **YYYY-MM-DD** — the last date a backwards-incompatible change was made. The version is **not** bumped for compatible additions.

State	Meaning	Revision	Significance
Draft	In-progress; not for use	2025-03-26	Streamable HTTP transport
Current	Ready; may get compatible changes	2025-06-18	Elicitation + structured output
Final	Past & frozen	2025-11-25	Current baseline — async Tasks

- ▶ Negotiation happens during initialization (the initialize exchange)
- ▶ Both sides MAY support multiple versions but MUST agree on one for the session
- ▶ **On mismatch: graceful termination** — the protocol provides error handling (deeper in Module 8)

Hands-On Lab — Observe the Protocol on the Wire

Goal: without writing any code, watch the \$5 lifecycle happen against real official MCP servers using the **MCP Inspector**.

- ▶ **Prereqs:** Node.js 18+ / npx, internet (first run), a browser
- ▶ **Launch against the filesystem server:**

```
npx -y @modelcontextprotocol/inspector npx -y @modelcontextprotocol/server-filesystem "$HOME/mcp-lab-sandbox"
```

- ▶ Connect → see the initialize handshake, negotiated protocolVersion & declared capabilities
- ▶ List & run a tool (read_file / list_directory) → inspect inputSchema and the tools/call round-trip
- ▶ Then the **everything** server → observe prompts, resources, logging & notifications (no id)

Steps 0 & 1: Creating a Sandbox & Launching Inspector

Action Checklist

Follow these steps to establish a safe process sandbox.

Step 0: Create a test folder and write a mock file. This creates a safe playground for the filesystem server.

Step 1: Execute the Inspector. We wrap the official Filesystem Server, providing the absolute path to the sandbox.

Why absolute paths: The filesystem server rejects relative folders to prevent accidental system access.

Launch URL: Open the local link printed in the terminal (e.g. `http://localhost:6274`).

terminal_commands.sh

```
## Linux/macOS Sandbox & Run
-----
mkdir -p "$HOME/mcp-lab-sandbox"
echo "Hello MCP!" > "$HOME/mcp-lab-sandbox/hello.txt"

npx -y @modelcontextprotocol/inspector \
  npx -y @modelcontextprotocol/server-filesystem \
    "$HOME/mcp-lab-sandbox"

## Windows PowerShell Sandbox & Run
-----
New-Item -ItemType Directory -Force \
  -Path "$HOME\mcp-lab-sandbox"
Set-Content -Path "$HOME\mcp-lab-sandbox\hello.txt" \
  -Value "Hello MCP!"

# Swap backslashes to forward slashes
$sandbox = "$HOME\mcp-lab-sandbox" -replace '\\', '/'
npx -y "@modelcontextprotocol/inspector" \
  npx -y "@modelcontextprotocol/server-filesystem" \
    "$sandbox"
```

Steps 2 & 3: Discovering, Calling, and Everything Server

Step 2: Discover and Execute Primitives

Inside the Inspector Web UI dashboard:

- **List tools:** Click 'List Tools' to issue ``tools/list``. Observe tools like ``read_file``, ``write_file``, and ``list_directory``.
- **Inspect Schema:** Look at ``read_file`` parameter rules.
- **Call Tool:** Run ``list_directory`` targeting your sandbox path.
- **Read File:** Execute ``read_file`` specifying ``<sandbox>/hello.txt``.
Verify your 'Hello MCP!' text is returned in the content panel.
- **Raw History:** Open the message pane and check the raw JSON-RPC packages side-by-side with slide templates.

Step 3: Test Multi-Primitive Everything Server

Press Ctrl+C to stop the filesystem server, then launch the 'everything' server:

```
$ npx -y @modelcontextprotocol/inspector \
```

```
npx -y @modelcontextprotocol/server-everything
```

- **Logging:** Run tools in the panel and watch notifications populate the 'Server Logs' panel.
- **Prompts:** Check 'Prompts' tab, list options, and get system prompt templates.
- **Notifications:** Inspect live event alerts that trigger without IDs.

Knowledge Check: Test Your Understanding

Q1: The N×M Scaling Challenge

Why do bespoke adapters fail to scale, and how does MCP resolve it?

Q3: Data & Transport Decoupling

Which layer chooses between STDIO and HTTP, and does the JSON-RPC packet change?

Q2: Participant Session Counts

If an MCP Host connects to 3 different servers, how many Client objects are spawned?

Q4: Discovering vs Calling

What JSON method discovers server tools, and what schema maps parameter rules?

Knowledge Check: Primitives & Methods

Q1: Model-Controlled Functions

Which MCP primitive exposes callable functions the LLM can invoke?

- A) Resources
- B) Tools
- C) Prompts
- D) Roots

Q3: Attachable Context

Which primitive provides user-selectable context blocks (files, DB rows, docs)?

- A) Tools
- B) Resources
- C) Prompts
- D) Sampling

Q2: Handshake Method

Which JSON-RPC method begins a new MCP session?

- A) tools/list
- B) ping
- C) initialize
- D) session/start

Q4: User-Invoked Templates

Which primitive surfaces user-triggered slash-command templates?

- A) Tools
- B) Resources
- C) Prompts
- D) Logging

Knowledge Check: Transport & Protocol

Q1: Local Default Transport

Which transport is the default for an MCP server running on the same machine as the host?

- A) WebSocket
- B) STDIO
- C) gRPC
- D) Streamable HTTP

Q3: Wire Format

What is the wire format for every MCP message, regardless of transport?

- A) Protobuf
- B) XML-RPC
- C) JSON-RPC 2.0
- D) MessagePack

Q2: Remote Server Transport

Which transport does MCP define for remote servers with streaming responses?

- A) Streamable HTTP
- B) Plain REST
- C) FTP
- D) Raw TCP

Q4: Correlating Request and Response

Which JSON-RPC field correlates a response back to its originating request?

- A) method
- B) jsonrpc
- C) id
- D) params

Knowledge Check: Capabilities & Safety

Q1: Capability Declaration

When do client and server declare which features they support?

- A) On first tool call
- B) During the initialize handshake
- C) When listing resources
- D) At process exit

Q3: Server-Initiated LLM Calls

Which capability lets a server request the client to run an LLM completion on its behalf?

- A) Roots
- B) Sampling
- C) Logging
- D) Resources

Q2: Consent Boundary

Which participant is responsible for getting user consent before executing a tool call?

- A) The Server
- B) The transport
- C) The Host
- D) The LLM directly

Q4: Live Tool Updates

Which notification tells the client a server's tool list has changed?

- A) tools/refresh
- B) notifications/tools/list_changed
- C) tools/reload
- D) initialize/again

Summary & What's Next

- ▶ MCP solves the **N×M integration problem** — “USB-C for AI”
- ▶ **Host** creates one **Client** per **Server** (1:1); servers are local (stdio) or remote (HTTP)
- ▶ Two layers: **Data** (JSON-RPC 2.0) inside **Transport** — same messages over any transport
- ▶ Primitives: servers expose **Tools / Resources / Prompts**; clients expose **Sampling / Elicitation / Logging**
- ▶ Lifecycle: **initialize** → **negotiate** → **discover** → **execute** → **notify**
- ▶ Versioning is date-based (**YYYY-MM-DD**); current baseline **2025-11-25**; mismatch → graceful termination

Next — Module 2: Everyday MCP I (Using MCP in Claude)

Move from observing the protocol to using it — configure servers in Claude Desktop & Claude Code.